CPS 3962 Object-Oriented Analysis and Design

Final Project: Server Operations Monitoring System

Dr. Hamza Djigal
Assistant Professor
Department of Computer Science, Wenzhou-Kean University
dhamza@kean.edu

March 26, 2026 (Spring)

Contents

1 Project Overview

Project Title: Server Operations Monitoring System

Objective: Develop a lightweight platform for registering monitored hosts, collecting operating-system metrics through a Java probe, storing historical performance data, and enabling secure remote operations from a browser-based console.

Key Features (MVP):

- **User Management and Authorization:** administrator and sub-account login, JWT session issuance, email verification, password reset, and host-level permission control.
- **One-Time Host Registration:** an administrator generates a short-lived registration token and a new probe uses it once to join the platform.
- **Real-Time Monitoring:** CPU, memory, disk, and network metrics are collected by the client and displayed in the dashboard with online/offline detection.
- **Historical Metrics:** runtime samples are persisted to InfluxDB and rendered in charts for trend analysis.
- **Browser-Based SSH:** the backend proxies WebSocket traffic to SSH so authorized users can open a terminal directly in the web UI.
- **Operations Governance:** Redis-based throttling, request tracing with Snowflake IDs, and Swagger/OpenAPI support for backend inspection.

2 Waterfall Methodology Phases

2.1 Requirements Gathering

Functional Requirements:

- **Administrator:** log in, issue registration tokens, view all hosts, rename hosts, edit node and location metadata, delete hosts, save SSH credentials, and create sub-accounts.
- **Sub-account:** log in, access only assigned hosts, inspect current and historical metrics, use the browser terminal when authorized, and maintain account security settings.
- **Probe Agent:** register to the server once, upload base hardware information, and push runtime metrics on a fixed schedule.
- **Platform Services:** send email verification codes, enforce throttling on sensitive actions, and expose monitoring APIs for the frontend.

Non-Functional Requirements:

- **Security:** JWT validation, role-based and per-host authorization, logout invalidation, and Redis throttling for repeated requests.
- **Performance:** the frontend refreshes host status every 10 seconds and the probe reports runtime data every minute.
- **Scalability:** relational metadata is separated from time-series data by using MySQL and InfluxDB for different workloads.

- **Reliability:** client cache refresh, graceful login validation, and persistent infrastructure services via Docker Compose.
- **Usability:** a Vue single-page application provides login, dashboard, charts, sub-user management, and SSH terminal access.

Deliverables:

- Use Case Diagram for Administrator, Sub-account, and Probe Agent
- Sequence Diagram for Host Registration and Monitoring Update
- Activity Diagram for Runtime Collection and Visualization Flow

2.2 System Analysis

The codebase reveals three core interaction scenarios:

- **Host onboarding:** the administrator requests `/api/monitor/register`, the probe sends the token to `/monitor/register`, then uploads hardware details through `/monitor/detail`.
- **Monitoring cycle:** the probe collects metrics with OSHI, Quartz triggers the upload job, the backend stores current values in memory and previous samples in InfluxDB, and the frontend polls for live and historical data.
- **Remote operations:** the user saves SSH settings, opens the terminal drawer, and the server upgrades to `/terminal/{clientId}` to proxy WebSocket traffic into an SSH shell.

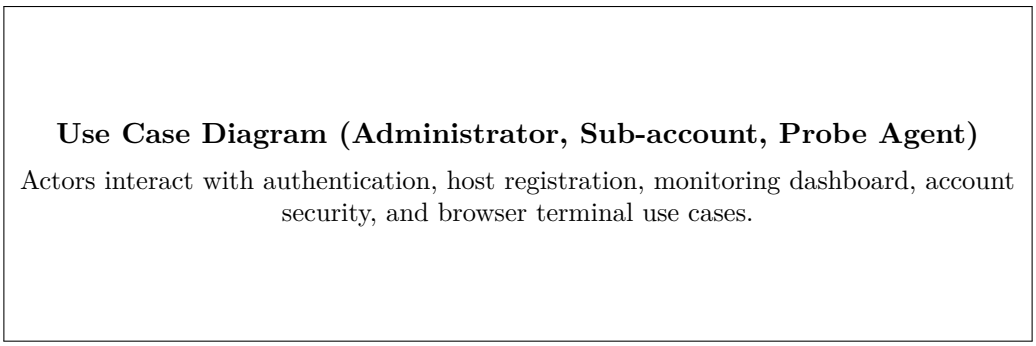

Figure 1: Use Case Diagram (Administrator, Sub-account, Probe Agent)

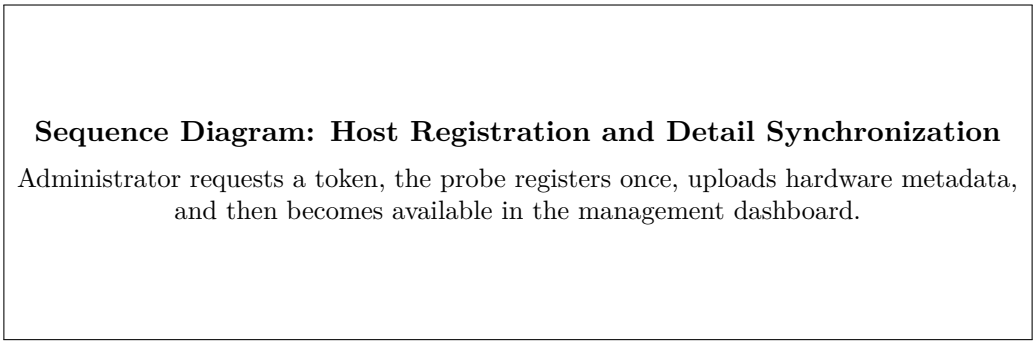

Figure 2: Sequence Diagram: Host Registration and Detail Synchronization

Activity Diagram: Runtime Monitoring Flow

Quartz triggers metric collection, the client measures CPU, memory, disk, and network usage, the backend stores data, and the UI visualizes updates.

Figure 3: Activity Diagram: Runtime Monitoring Flow

2.3 System Design

Architecture Summary:

- `monitor_server/`: Spring Boot backend with Spring Security, MyBatis-Plus, Redis, RabbitMQ, InfluxDB, and WebSocket-to-SSH proxying.
- `monitor_client/`: Java probe using OSHI for hardware and runtime metrics, Quartz for scheduling, and Java HTTP client for reporting.
- `monitor-web/`: Vue 3 frontend using Element Plus, Pinia, Axios, ECharts, and xterm.js.

Class Diagram (Design Level)

Key design classes include Account, Client, ClientDetail, ClientSsh, RuntimeDetailVO, JwtUtils, FlowUtils, InfluxDbUtils, and TerminalWebSocket.

Figure 4: Class Diagram (Design Level)

Database Schema

Table / Store	Columns	Data Type	Description
db_account	id, username, email, password, role, clients, register_time	INT, VARCHAR, JSON, DATETIME	Stores administrator and sub-account identities, encrypted passwords, and assigned host IDs.
db_client	id, name, token, location, node, register_time	INT, VARCHAR, DATETIME	Stores registered hosts and their onboarding metadata.
db_client_detail	client_id, os_arch, os_name, os_version, os_bit, cpu_name, cpu_core, memory, disk, ip	INT, VARCHAR, DOUBLE	Stores the latest hardware profile for each monitored host.
db_client_ssh	id, port, username, password	INT, VARCHAR	Stores SSH connection settings keyed by host ID.
InfluxDB runtime	clientId, cpuUsage, memoryUsage, diskUsage, networkUpload, networkDownload, diskRead, diskWrite, timestamp	Time-series measurement fields	Stores runtime samples for history charts and trend analysis.

Table 1: Primary Data Storage Design

UI Wireframes and Main Screens

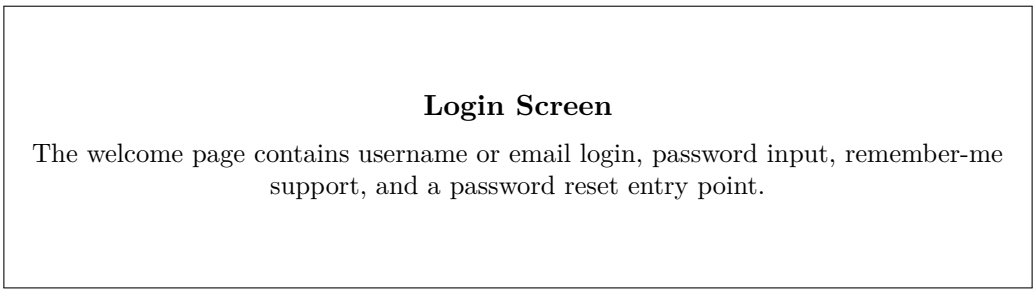

Figure 5: Login Screen

Figure 6: Host Management Dashboard

Host Detail and SSH Terminal Screen

The detail panel combines host metadata, current performance dashboards, history charts, node editing, and terminal launch for remote operations.

Figure 7: Host Detail and SSH Terminal Screen

2.4 Implementation

Implemented Modules:

1. **Authentication and Account Security**
   Spring Security handles form login at `/api/auth/login`; successful authentication creates a JWT through `JwtUtils`. Password reset, email modification, and verification-code delivery are managed by `AuthorizeController` and `AccountServiceImpl`.

2. **Host Registration and Metadata Synchronization**
   `ClientServiceImpl` generates one-time registration tokens, creates a new host record after validation, and updates hardware details received from the probe.

3. **Runtime Monitoring Pipeline**
   The client uses OSHI to collect metrics, Quartz schedules the job every minute, and the backend caches current runtime values while writing previous snapshots to InfluxDB for historical retrieval.

4. **Frontend Dashboard and Data Visualization**
   Vue components such as `PreviewCard.vue`, `ClientDetails.vue`, and `RuntimeHistory.vue` provide live host cards, detail drawers, and ECharts-based monitoring charts.

5. **Browser-Based SSH**
   `TerminalWindow.vue` collects SSH credentials, and `TerminalWebSocket.java` uses JSch to bridge browser traffic to the host shell.

6. **Cross-Cutting Infrastructure**
   Redis throttling is implemented in `FlowUtils`, request logs are correlated with `SnowflakeIdGenerator`, email verification is sent through RabbitMQ, and Swagger is enabled for API inspection.

Object-Oriented Design Characteristics:

- **Encapsulation:** account, host, SSH, and runtime entities are represented as DTO and VO classes with clear responsibilities.
- **Abstraction:** controllers expose clean API boundaries while services encapsulate security, registration, cache, and persistence logic.
- **Modularity:** backend, probe, and frontend are separated into independent deployable modules.
- **Reusability:** utility classes such as `JwtUtils`, `FlowUtils`, and `MonitorUtils` centralize repeated behavior.

2.5 Testing

Repository-Provided Automated Tests:

- `JwtUtilsTest`: verifies that JWT blacklist expiration is written to Redis using millisecond precision.
- `MonitorServerApplicationTests`: validates that the Spring Boot backend context loads successfully.
- `MonitorClientApplicationTests`: validates that the client application context loads successfully.

Build Verification Performed for This Report:

- **Frontend:** `npm run build` succeeds and generates a production bundle under `monitor-web/dist/`.
- **Backend:** Maven dependency resolution works, but compilation currently fails in this environment with `TypeTag :: UNKNOWN` during the server build. This indicates a toolchain compatibility issue that should be resolved before final deployment.
- **Client:** the Spring Boot test starts successfully but blocks on the interactive registration prompt because `ServerConfiguration` requests console input when no saved configuration file exists. This should be mocked or disabled for CI-style testing.

Planned Integration and System Tests:

- Login, logout, and JWT invalidation flow
- Email verification and password reset flow
- One-time host registration and hardware detail upload
- Runtime metric polling and InfluxDB history retrieval
- Per-host authorization for sub-accounts
- SSH credential save and browser terminal session establishment

Test Item	Expected Result	Evidence Source
Backend context startup	Spring Boot server loads without bean wiring errors	Existing automated test
Client context startup	Probe application starts with configured beans	Existing automated test
JWT blacklist expiration	Logout token invalidation uses correct Redis expiration unit	Existing automated test
Frontend production build	Vue application bundles successfully for deployment	Executed build verification
Host registration flow	A valid one-time token creates exactly one new host record	Controller and service implementation
Runtime history query	Historical samples are returned from InfluxDB for the selected host	`InfluxDbUtils.readRuntimeD`
SSH terminal access	Authorized user can open terminal for allowed hosts only	WebSocket and filter implementation

Table 2: Testing Plan and Current Verification Basis

2.6 Deployment

- **Local Development:** run the backend in `monitor_server/`, the frontend in `monitor-web/`, and the client in `monitor_client/`. Vite can point to the backend through `VITE_API_BASE`.
- **Containerized Deployment:** the repository includes `docker-compose.yml` for MySQL, Redis, RabbitMQ, InfluxDB, MinIO, backend, and frontend services.
- **Production Considerations:** configure HTTPS, restrict CORS origins, rotate secrets, replace plaintext development credentials, and place the frontend and backend behind a reverse proxy that supports WebSocket upgrade.
- **Operational Notes:** MySQL stores metadata, InfluxDB stores runtime history, Redis supports throttling and token state, and RabbitMQ decouples email verification delivery.

Documentation and Final Submission

- Requirements specification aligned with code modules
- UML deliverables: use case, sequence, activity, and class diagrams
- Database schema for relational and time-series storage
- UI screen descriptions for login, dashboard, detail, and terminal views
- Test plan and current repository verification evidence

- Deployment notes based on local and Docker Compose execution

3 Optional Advanced Features (Extra Credit)

- Threshold-based alerting with email or webhook notifications
- SSH key authentication and secret vault integration
- Anomaly detection for CPU, memory, or network behavior
- Retention-policy management and long-term trend reporting
- Multi-tenant audit log export and approval workflow for sensitive operations

Suggested Tech Stack:

- **Frontend:** Vue 3, Vite, Element Plus, Pinia, ECharts, xterm.js
- **Backend:** Java Spring Boot, Spring Security, MyBatis-Plus, WebSocket
- **Database:** MySQL and InfluxDB
- **Infrastructure:** Redis, RabbitMQ, MinIO, Docker Compose
- **Probe Agent:** Java 17, OSHI, Quartz